

Relay

An On-Device, Voice-Controlled, Screen-Reading Accessibility Agent

Kartik Pandey

IIIT Naya Raipur

Snapdragon AI Lab Build & Present Challenge, September 2026

Outline

Introduction

Pipeline Overview

Screen Understanding

The Guardrail Engine

Implementation

Results

Limitations & Future Work

The Problem

- People with **limited hand mobility or low vision** can't reliably operate a Windows PC through a mouse and a screen alone.
- Existing screen readers *describe* a screen but don't act on it; existing automation agents act, but weren't built around this population's safety needs.
- **Idea:** a voice-first agent that both understands the screen and acts on it — with a person narrating every step, and a spoken confirmation gate on anything consequential.

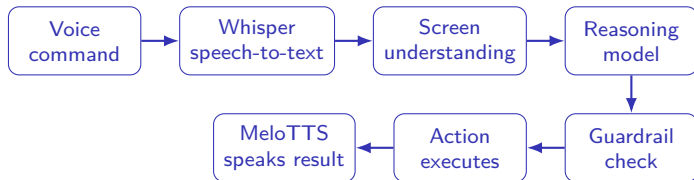
Design goals

On-device only, no cloud round-trip for speech or screen understanding · never act on anything sensitive without a spoken “yes” · scoped assistive technology, not a general autonomous agent

Why Not Just Use a General Computer-Use Agent?

- General autonomous agents can already read a screen and click or type on it.
- But the cost of being wrong is asymmetric:
 - Agent mis-clicks a harmless button → annoying, but recoverable.
 - Agent confidently taps “Send” / “Delete” / “Pay” on a misread screen → real, sometimes irreversible harm.
- ⇒ the whole design is biased toward **safety over autonomy**: nothing consequential executes without a spoken confirmation, ever.

End-to-End Pipeline



- Screen understanding checks the UI Automation accessibility tree first; only falls back to a vision-language model when that tree doesn't have enough named, enabled controls to work with.
- The guardrail check never changes: two-tier ALLOW / CONFIRM / BLOCK against pattern-matched control names, evaluated *before* any action executes.

The Fast Path — UI Automation Tree

- Reads the live accessibility tree first: named, enabled controls, no model call needed.
- Verified against real, live windows on this machine:
 - **Media Player** — 211 elements, 113 named + enabled, real actionable content (Search, Open file(s), Recent media).
 - **Notepad** — a simple-app proxy, 24 elements, a real scroll action executed successfully.
- Falls back to a vision-language model only when the tree genuinely doesn't have enough to work with — not the common case, the exception.

The Chromium Gap — Found, Verified, and Fixed

- **Edge and Outlook** are both Chromium/WebView2-based: walking their UI Automation tree 12–14 levels deep surfaces window chrome only — **zero** real page content, confirmed, not assumed.
- **Fixed for Edge** via the Chrome DevTools Protocol: real page content read from a live Wikipedia page, a real value typed into the real search box, and the write independently confirmed by re-reading the live DOM afterward.
- Outlook's equivalent gap remains open — it's a desktop app, not a browser tab the DevTools Protocol can attach to.

Every claim here is labeled by how it was verified

Run on real hardware/software, confirmed from real source code, or explicitly marked as not yet possible without Snapdragon-class hardware.

Two-Tier Decision

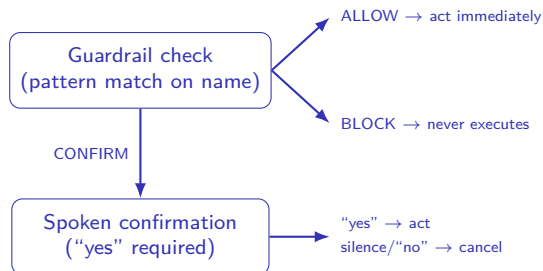
- Every decision is checked against the target control's name *before* it executes:

$$\text{decision} = \begin{cases} \text{BLOCK} & \text{name matches a block pattern (irreversible)} \\ \text{CONFIRM} & \text{name matches a confirm pattern (consequential)} \\ \text{ALLOW} & \text{otherwise} \end{cases}$$

- CONFIRM: send/submit/post/publish, pay/buy/transfer, delete/remove/erase, account or security changes — requires a spoken “yes” before proceeding.
- BLOCK: factory reset, reformat, disabling the firewall or antivirus — never executes, no override.
- Every decision is written to a persisted, append-only audit log.

- 18 adversarial test cases built from plausible real controls across all three target apps.
- Found a genuine gap: the pattern `\bdelete\b`'s word boundary didn't match inside "deleted" — so a real control like "Empty deleted items folder" silently fell through to `ALLOW`.
- Fixed by widening `delete/remove/erase` to also catch `-ed/-ing/-ion` forms — a widening, never a narrowing, by design: these patterns should get harder to change, not easier.

The Full Decision Flow



- Most decisions resolve immediately (ALLOW or BLOCK).
- Only genuinely consequential actions pay the extra cost of a spoken confirmation round-trip.

What's Actually Real

- **Whisper speech-to-text** — real transcription of a known test clip, reproduced three independent ways.
- **MeloTTS-EN** — real synthesis, real audio, played back through real speakers (five independent real blockers found and fixed to get there).
- **The full `agent.py` orchestration loop** — a real mic recording → real transcription → real UI Automation tree read → real guardrail gate → real action → real speech, proven end to end against a live window. Only the NPU-dependent reasoning step is mocked.
- **`cdp_perception.py`** — the Chrome DevTools Protocol fix for Edge, wired into `agent.py`'s real orchestration and demonstrated live.

Competitive Differentiation

Project	Safety mechanism	Enforcement
Deft	None found anywhere in either backing repo	N/A
Orbit	None — but never takes actions at all	N/A
ScreenSaathi	Keyword list downgrades risky plans	Code-level
Sally	A prompt asking the model nicely	Model-enforced only
Relay	ALLOW / CONFIRM / BLOCK + spoken yes + audit log	Code-level, pre-execution

Table: Verified against each project's real, public source code — not a secondhand summary.

- Of five comparable projects, Relay is the only one with a deterministic, code-level safety gate that doesn't depend on a language model correctly following an instruction.

Metric	Value
Whisper-base encoder	~49.1 ms
Whisper-base decoder	~3.7 ms

Table: Compiled and profiled on a real Snapdragon X Elite CRD via Qualcomm AI Hub's hosted device farm.

- A real compile + profile job on actual target silicon — not a simulation, and an order of magnitude faster than this laptop's CPU.
- Precisely scoped: this is AI Hub's compile/profile path, not yet a full GenieX runtime measurement, which needs physical device access.

- **Open:** Outlook's Chromium accessibility gap — the same limitation as Edge, but it's a desktop app the DevTools Protocol can't attach to. Unsolved, documented as such.
- **Open:** full GenieX-runtime, on-device latency for the reasoning model — needs physical Snapdragon hardware, not yet measured.
- **Done:** 18 adversarial guardrail tests, a fixed task-list harness (100% pass rate, 5/5), and short-term reasoning memory (last 3 turns).
- This submission was built without physical Snapdragon hardware — every claim in this deck is labeled by exactly how it was verified.

github.com/kartik1pandey/relay-accessibility-agent
